

MATERIAL COMPLEMENTARIO: UNIDAD 7

A continuación, presentamos un resumen de los conceptos clave extraídos del libro *Ingeniería del Software: Un Enfoque Práctico* (Roger S. Pressman, 7ma Edición), seleccionados específicamente para complementar la Unidad 7.

CAPÍTULO 17: Estrategias de Prueba de Software

El testing es un conjunto de actividades que se pueden planificar por adelantado y realizar sistemáticamente. Pressman define una estrategia de prueba de software como un mapa que integra el diseño de casos de prueba con la organización de los pasos planificados.

1. Verificación y Validación (V&V)

A menudo se confunden, pero Pressman (citando a Boehm) establece una diferencia crítica:

- **Verificación:** Se refiere al conjunto de actividades que aseguran que el software implementa correctamente una función específica.
 - *La pregunta clave:* "¿Estamos construyendo el producto **correctamente?**"
- **Validación:** Se refiere a un conjunto diferente de actividades que aseguran que el software construido se ajusta a los requisitos del cliente.
 - *La pregunta clave:* "¿Estamos construyendo el producto **correcto?**"

La calidad no se logra solo con testing; se debe integrar la calidad en el software durante todo el proceso de ingeniería.

2. La Espiral de las Pruebas

Pressman visualiza el proceso de pruebas como una espiral que avanza desde el centro (código) hacia el exterior (sistema):

1. **Prueba de Unidad:** Comienza en el vórtice de la espiral. Se concentra en cada unidad del software (componente o clase) tal como se implementa en el código fuente. Utiliza intensivamente técnicas de **Caja Blanca**.
2. **Prueba de Integración:** A medida que nos movemos hacia afuera, el foco cambia a diseño y construcción de la arquitectura. Se abordan los problemas asociados con la verificación y la construcción del programa.

3. **Prueba de Validación:** Se enfoca en los requisitos. Aquí se valida que el software cumpla con los requerimientos establecidos por el cliente. Utiliza técnicas de **Caja Negra**.
4. **Prueba del Sistema:** En la capa más externa, el software y otros elementos del sistema se prueban como un todo. Verifica que todos los elementos (hardware, personas, bases de datos) engranen adecuadamente.

CAPÍTULO 18: Técnicas de Prueba (Caja Blanca y Caja Negra)

Mientras que la estrategia (Cap. 17) define el "plan de batalla", las tácticas (Cap. 18) definen cómo diseñar los casos de prueba específicos.

A. Pruebas de Caja Blanca (White-Box Testing)

También llamada prueba de caja de cristal. Es una filosofía de diseño de casos de prueba que utiliza la estructura de control descrita en el diseño procedimental para derivar casos de prueba.

El objetivo es garantizar que:

1. Se ejerciten todos los caminos independientes de cada módulo al menos una vez.
2. Se ejerciten todas las decisiones lógicas en sus lados verdadero y falso.
3. Se ejecuten todos los bucles en sus límites y dentro de sus límites operativos.
4. Se ejerciten las estructuras de datos internas para asegurar su validez.

La Prueba del Camino Básico (Basis Path Testing): Es una técnica de caja blanca que permite al diseñador obtener una medida de la complejidad lógica de un diseño procedimental y usar esa medida como guía para definir un conjunto básico de rutas de ejecución.

B. Pruebas de Caja Negra (Black-Box Testing)

Las pruebas de caja negra, también llamadas pruebas de comportamiento, se enfocan en los requisitos funcionales del software.

Diferencia Fundamental: La prueba de caja negra **no es una alternativa** a la técnica de caja blanca. Es un enfoque **complementario** que intenta descubrir una clase diferente de errores. Mientras la caja blanca se enfoca en la lógica interna, la caja negra intenta encontrar:

1. Funciones incorrectas o ausentes.
2. Errores de interfaz.
3. Errores en estructuras de datos o en accesos a bases de datos externas.
4. Errores de comportamiento o de rendimiento.
5. Errores de inicialización y terminación.

A diferencia de la caja blanca (que se realiza temprano), la caja negra tiende a aplicarse durante etapas posteriores de la prueba.

C. Técnicas de Diseño de Caja Negra

Pressman detalla pautas específicas para implementar estas técnicas:

1. Partición de Equivalencia Divide el dominio de entrada de un programa en clases de datos de las que se pueden derivar casos de prueba. Se busca definir un caso de prueba que descubra clases de errores, reduciendo así el número total de casos necesarios.

Pautas para definir clases:

- Si una condición de entrada especifica un **rango** (ej. de 1 a 999), se definen una clase válida y dos inválidas (ej. <1 y >999).
- Si una condición de entrada requiere un **valor específico**, se define una clase válida y dos inválidas.
- Si una condición de entrada especifica un **miembro de un conjunto** (ej. "Auto, Camión, Moto"), se define una clase válida por cada miembro y una clase inválida (algo que no esté en el conjunto).
- Si una condición de entrada es **booleana**, se define una clase válida y una inválida.

2. Análisis de Valores Límite (BVA) Un gran número de errores ocurre en los límites del dominio de entrada más que en el centro. El BVA es una técnica que lleva a la elección de casos de prueba que ejerciten los valores extremos.

Pautas para el BVA:

1. Si una condición de entrada especifica un rango delimitado por los valores a y b , se deben probar los valores a y b (límites exactos), y también valores justo por encima y justo por debajo de a y b .
2. Si una condición de entrada especifica un número de valores (ej. "la lista puede tener hasta 10 ítems"), se deben probar los casos con 0 ítems, 1 ítem, y 10 ítems (máximo) y 11 ítems (máximo + 1).
3. Aplicar las pautas 1 y 2 a las condiciones de **salida** (ej. si el sistema debe generar una tabla de máximo 5 filas, intentar forzar que genere 6).

Referencia: Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7a ed.). McGraw-Hill.